

Agent Phantom Recovery

11-database-schema.md

Production Database Schema

Version: 1.0

1. Purpose

This document defines the production database architecture and schema for Agent Phantom Recovery.

Unlike the Backend Schema document, which describes logical entities and service contracts, this document defines:

- Physical databases
- Tables
- Relationships
- Indexes
- Constraints
- Storage layers
- Vector storage
- Caching architecture
- Audit storage
- Event persistence

The database architecture must support:

- Long-horizon execution
 - Multi-agent workflows
 - Repository intelligence
 - Memory persistence
 - Event sourcing
 - Tool execution tracking
 - Antigravity IDE synchronization
-

2. Database Philosophy

The platform is not a CRUD application.

Most AI products store:

```
User
↓
Conversation
↓
Message
```

Agent Phantom Recovery stores:

```
User
↓
Project
↓
Repository
↓
Task
↓
Execution
↓
Evidence
↓
Finding
↓
Patch
↓
Verification
↓
Report
```

The database is therefore execution-centric.

3. Storage Architecture

The platform should use multiple storage systems.

PostgreSQL

Primary transactional database.

Stores:

- users
- projects
- tasks
- plans
- reports

- executions
 - metadata
-

Vector Database

Stores:

- memories
- embeddings
- repository chunks
- semantic indexes

Recommended:

Qdrant

Alternative:

Weaviate

Redis

Stores:

- sessions
 - cache
 - queues
 - temporary state
-

Object Storage

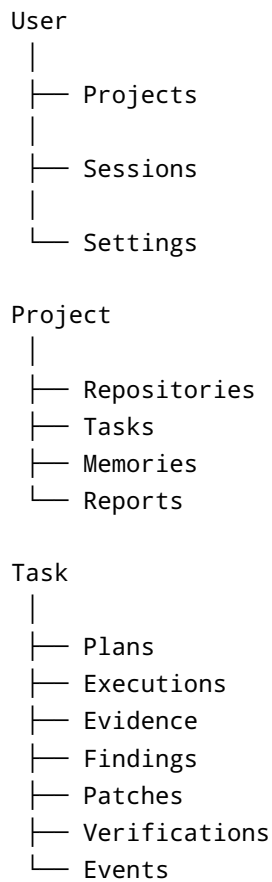
Stores:

- reports
- screenshots
- repository artifacts
- logs
- exported files

Recommended:

S3 Compatible Storage

4. Database Relationship Overview



5. Users Table

users

Column	Type
id	UUID
email	VARCHAR
username	VARCHAR
password_hash	TEXT
role	VARCHAR
status	VARCHAR
created_at	TIMESTAMP

Column	Type
updated_at	TIMESTAMP

Indexes

```
idx_users_email
idx_users_username
```

6. User Sessions Table

```
user_sessions
```

Column	Type
id	UUID
user_id	UUID
access_token	TEXT
refresh_token	TEXT
expires_at	TIMESTAMP
created_at	TIMESTAMP

Purpose

Supports:

- login
- session persistence
- multi-device access

7. Projects Table

```
projects
```

Column	Type
id	UUID
owner_id	UUID
name	VARCHAR
description	TEXT
visibility	VARCHAR
created_at	TIMESTAMP

Relationships

User
↓
Projects

One user may own many projects.

8. Repositories Table

repositories

Column	Type
id	UUID
project_id	UUID
source_url	TEXT
branch	VARCHAR
language	VARCHAR
framework	VARCHAR
file_count	INTEGER
created_at	TIMESTAMP

Purpose

Stores repository metadata.

Not source code itself.

9. Repository Index Table

repository_indexes

Column	Type
id	UUID
repository_id	UUID
symbol_count	INTEGER
dependency_count	INTEGER
indexed_at	TIMESTAMP

Purpose

Stores indexing metadata.

Used by repository intelligence service.

10. Tasks Table

The most important table.

tasks

Column	Type
id	UUID
project_id	UUID
goal	TEXT
status	VARCHAR
priority	INTEGER
current_plan_id	UUID
created_at	TIMESTAMP
updated_at	TIMESTAMP

Task States

```
CREATED
PLANNING
COLLECTING_EVIDENCE
ANALYZING
EXECUTING
VERIFYING
COMPLETED
FAILED
BLOCKED
```

Indexes

```
idx_tasks_project
idx_tasks_status
idx_tasks_priority
```

11. Plans Table

```
plans
```

Column	Type
id	UUID
task_id	UUID
version	INTEGER
status	VARCHAR
summary	TEXT
created_at	TIMESTAMP

12. Plan Steps Table

```
plan_steps
```


Column	Type
id	UUID
plan_id	UUID
step_order	INTEGER
description	TEXT
tool	VARCHAR
status	VARCHAR

Example

```
Run Tests
Inspect Logs
Analyze Module
Generate Patch
Verify
```

13. Evidence Table

```
evidence
```

Column	Type
id	UUID
task_id	UUID
source	VARCHAR
type	VARCHAR
content	TEXT
confidence	FLOAT
created_at	TIMESTAMP

Evidence Types

```
LOG
TEST
```

FILE
CODE
TERMINAL
BROWSER
SEARCH
MEMORY

14. Findings Table

findings

Column	Type
id	UUID
task_id	UUID
summary	TEXT
severity	VARCHAR
confidence	FLOAT
status	VARCHAR

Purpose

Represents validated issues discovered during analysis.

15. Finding Evidence Table

finding_evidence

Column	Type
finding_id	UUID
evidence_id	UUID

Purpose

Many-to-many mapping.

One finding may reference many evidence items.

16. Patches Table

patches

Column	Type
id	UUID
task_id	UUID
finding_id	UUID
summary	TEXT
status	VARCHAR
created_at	TIMESTAMP

Patch States

DRAFT
GENERATED
TESTED
VERIFIED
REJECTED

17. Patch Files Table

patch_files

Column	Type
id	UUID
patch_id	UUID
file_path	TEXT
change_summary	TEXT

Purpose

Tracks modified files.

18. Verification Table

verifications

Column	Type
id	UUID
task_id	UUID
patch_id	UUID
status	VARCHAR
confidence	FLOAT
created_at	TIMESTAMP

19. Verification Checks Table

verification_checks

Column	Type
id	UUID
verification_id	UUID
check_type	VARCHAR
result	VARCHAR

Examples

TESTS
PATCH_REVIEW
REGRESSION_CHECK
VERIFIER_APPROVAL

20. Reports Table

reports

Column	Type
id	UUID
task_id	UUID
report_type	VARCHAR
storage_path	TEXT
created_at	TIMESTAMP

Purpose

Stores generated deliverables.

21. Executions Table

Tracks every action.

executions

Column	Type
id	UUID
task_id	UUID
action	TEXT
status	VARCHAR
started_at	TIMESTAMP
completed_at	TIMESTAMP

Importance

Critical for:

- observability

- recovery
- auditing

22. Checkpoints Table

checkpoints

Column	Type
id	UUID
task_id	UUID
snapshot	JSONB
created_at	TIMESTAMP

Purpose

Supports:

- recovery
- resume execution
- long-horizon tasks

23. Events Table

events

Column	Type
id	UUID
task_id	UUID
event_type	VARCHAR
payload	JSONB
created_at	TIMESTAMP

Example Events

```
TaskCreated
TaskStarted
ToolExecuted
VerificationPassed
```

24. Tool Executions Table

```
tool_executions
```

Column	Type
id	UUID
task_id	UUID
tool_name	VARCHAR
status	VARCHAR
duration_ms	INTEGER
created_at	TIMESTAMP

Purpose

Tracks tool usage.

25. Terminal Sessions Table

```
terminal_sessions
```

Column	Type
id	UUID
task_id	UUID
environment_id	UUID
created_at	TIMESTAMP

26. Terminal Commands Table

terminal_commands

Column	Type
id	UUID
session_id	UUID
command	TEXT
output	TEXT
exit_code	INTEGER
created_at	TIMESTAMP

Antigravity Mapping

Directly powers:

Terminal Workspace

27. Browser Sessions Table

browser_sessions

Column	Type
id	UUID
task_id	UUID
current_url	TEXT
created_at	TIMESTAMP

28. Browser Observations Table

browser_observations

Column	Type
id	UUID
session_id	UUID
observation	TEXT
created_at	TIMESTAMP

Antigravity Mapping

Directly powers:

Browser Workspace

29. Memory Database Architecture

Memory should not live entirely in PostgreSQL.

Memory Metadata Table

memory_records

Column	Type
id	UUID
project_id	UUID
type	VARCHAR
importance	FLOAT
created_at	TIMESTAMP

Memory Types

WORKING
SESSION
PROJECT
EXPERIENCE

30. Vector Memory Collections

Stored in Qdrant.

Working Memory Collection

working_memory

Session Memory Collection

session_memory

Project Memory Collection

project_memory

Experience Memory Collection

experience_memory

Stored Fields

```
{
  "id": "",
  "project_id": "",
  "content": "",
  "embedding": [],
  "importance": 0.0
}
```

31. Repository Embedding Collections

Separate collection.

```
repository_chunks
```

Purpose

Stores:

- code chunks
- file summaries
- architecture summaries

for semantic retrieval.

32. Redis Schema

Used for transient data.

Cache Keys

```
task:{id}  
plan:{id}  
memory:{id}
```

Queue Keys

```
planning_queue  
analysis_queue  
verification_queue  
execution_queue
```

Session Keys

```
session:{id}
```

33. Audit Tables

Audit Logs

audit_logs

Column	Type
id	UUID
actor_id	UUID
action	TEXT
metadata	JSONB
created_at	TIMESTAMP

Purpose

Tracks:

- repository access
- tool usage
- memory changes
- permission changes

34. Performance Indexes

Required indexes:

tasks(status)
tasks(project_id)

evidence(task_id)

findings(task_id)

patches(task_id)

executions(task_id)

events(task_id)

```
tool_executions(task_id)
```

These will become critical at scale.

35. Antigravity IDE Data Mapping

Backend tables directly power IDE surfaces.

Browser Panel

```
browser_sessions  
browser_observations
```

Terminal Panel

```
terminal_sessions  
terminal_commands
```

Source Code Panel

```
repositories  
repository_indexes  
repository_chunks
```

Timeline Panel

```
events  
executions
```

Memory Panel

```
memory_records  
vector memory collections
```

Chat / Agent Panel

tasks
plans
findings
reports

36. Database Design Principles

The schema follows:

Task-Centric Design

Everything belongs to a task.

Evidence-Centric Reasoning

Findings must connect to evidence.

Verification-Driven Architecture

Nothing is complete without verification.

Long-Horizon Persistence

Tasks survive restarts.

Observability First

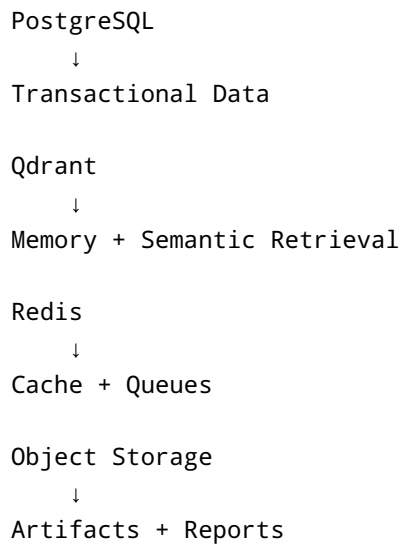
Every action is traceable.

Antigravity Integration

Every major workspace surface maps directly to stored entities.

37. Final Database Schema Summary

The production database architecture consists of:



The schema is optimized for:

- Long-horizon execution
- Multi-agent workflows
- Repository intelligence
- Memory retrieval
- Verification pipelines
- Embedded browser support
- Embedded terminal support
- Antigravity IDE workspace synchronization

and serves as the persistence foundation for the entire Agent Phantom Recovery platform.